Środowisko: Arduino IDE, Arduino UNO z nakładką multifunkcyjną (4 diody, przyciski, buzzer, 4-cyfrowy wyświetlacz 7-segmentowy), biblioteka Arduino_FreeRTOS.
Związek z wykładem: wykład 3 — przerwania, semafory binarne i liczące, mutex, inwersja priorytetów, kolejki.

Nakładka multifunkcyjna — połączenia

Element	Pin Arduino	Uwagi
diody D1–D4	13, 12, 11, 10	aktywne stanem niskim (LOW = świeci)
przyciski S1–S3	A1, A2, A3	aktywne stanem niskim, podciągnięte na płytce
buzzer	3	aktywny stanem niskim
wyświetlacz (74HC595)	LATCH 4, CLK 7, DATA 8	multipleksowany — trzeba go stale odświeżać

Wyświetlacz odświeża przerwanie Timer2 (co 2 ms jedna cyfra) — zadania wpisują jedynie zawartość bufora. Poniższy kod jest wspólny dla zadań 3 i 4:

```
#include <Arduino_FreeRTOS.h>
#include <semphr.h>
#include <queue.h>

const byte LATCH = 4, CLK = 7, DATA = 8;
const byte SEG[] = {0xC0,0xF9,0xA4,0xB0,0x99,0x92,0x82,0xF8,0x80,0x90};
const byte DIG[] = {0xF1,0xF2,0xF4,0xF8};
volatile byte bufor[4] = {0, 0, 0, 0};          // cyfry do wyświetlenia

ISR(TIMER2_COMPA_vect) {                        // co 2 ms jedna cyfra
  static byte d = 0;
  digitalWrite(LATCH, LOW);
  shiftOut(DATA, CLK, MSBFIRST, SEG[bufor[d]]);
  shiftOut(DATA, CLK, MSBFIRST, DIG[d]);
  digitalWrite(LATCH, HIGH);
  d = (d + 1) & 3;
}

void displayInit() {
  pinMode(LATCH, OUTPUT); pinMode(CLK, OUTPUT); pinMode(DATA, OUTPUT);
  TCCR2A = (1 << WGM21);                        // tryb CTC
  TCCR2B = (1 << CS22) | (1 << CS21);           // preskaler 256
  OCR2A  = 124;                                 // 16 MHz / 256 / 125 = 500 Hz
  TIMSK2 = (1 << OCIE2A);
}

void wyswietl(int v) {                          // wpis liczby do bufora
  for (int i = 3; i >= 0; i--) { bufor[i] = v % 10; v /= 10; }
}
```

Zadanie 1: Semafor binarny — sygnalizacja naciśnięcia przycisku

Etap A — odpytywanie przycisku w zadaniu

1. W setup() utwórz semafor binarny: sem = xSemaphoreCreateBinary();

2. Utwórz zadanie ButtonTask, które co 20 ms sprawdza S1 (digitalRead) i po wykryciu zbocza (zmiana z HIGH na LOW) wykonuje xSemaphoreGive(sem).
3. Utwórz zadanie LedTask, które czeka na semafor (xSemaphoreTake(sem, portMAX_DELAY)) i zapala D1 na 200 ms.
4. Sprawdź: dioda reaguje tylko wtedy, gdy ButtonTask sygnalizuje zdarzenie.

Etap B — przerwanie od przycisku (odroczona obsługa przerwania)

5. Zastąp zadanie ButtonTask przerwaniem od zmiany stanu pinu A1 (PCINT). Procedura obsługi przerwania ma tylko oddać semafor funkcją xSemaphoreGiveFromISR().
6. Dodaj prostą eliminację drgań styków: ignoruj zbocza w odstępie krótszym niż 50 ms.

```
ISR(PCINT1_vect) {                          // zmiana stanu A1..A3
  static uint32_t ostatnie = 0;
  if (digitalRead(A1) == LOW && millis() - ostatnie > 50) {
    ostatnie = millis();
    xSemaphoreGiveFromISR(sem, NULL);       // tylko sygnał
  }
}

// w setup():
PCICR  |= (1 << PCIE1);                     // przerwania portu C
PCMSK1 |= (1 << PCINT9);                    // pin A1
```

> **Uwaga:** w tej wersji portu (UNO) zadanie odblokowane z przerwania zostanie uruchomione najpóźniej przy następnym ticku (do ok. 16 ms). Na ESP32 natychmiastowe przełączenie zapewnia portYIELD_FROM_ISR() — patrz wykład 3.

Do analizy:

- Co się dzieje, gdy naciskamy przycisk szybciej, niż LedTask zdąży odebrać sygnał? Czy sygnały się „kolejkują”?
- Porównaj etap A i B: zużycie procesora i czas reakcji.

Zadanie 2: Semafor liczący — sygnalizacja wielu zdarzeń

1. Utwórz semafor liczący z zakresem 0..5: semCount = xSemaphoreCreateCounting(5, 0);
2. Utwórz zadanie ButtonProducer, które przy każdym naciśnięciu S1 albo S2 wykonuje xSemaphoreGive(semCount).
3. Utwórz zadanie BuzzerConsumer, które czeka na semafor i po otrzymaniu sygnału krótko „piknie” buzzerem (100 ms), a następnie odczeka 200 ms.
4. Naciśnij kilkukrotnie szybko przycisk: buzzer powinien odtworzyć wszystkie sygnały z kolejki.

Do analizy:

- Co się stanie, gdy licznik semafora osiągnie maksymalną wartość 5? Jaką wartość zwraca wtedy xSemaphoreGive()?
- Semafor liczący nie chroni pojedynczego zasobu przed jednoczesnym dostępem — do czego więc służy? (wskazówka: pula N jednakowych zasobów, zliczanie zdarzeń).

Zadanie 3: Mutex — ochrona wyświetlacza i inwersja priorytetów

Sytuacja: dwa zadania korzystają z jednego wyświetlacza 7-segmentowego. Bez synchronizacji cyfry jednego zadania nadpisują cyfry drugiego.

Etap A — mutex

1. Utwórz mutex: dispMutex = xSemaphoreCreateMutex();
2. Utwórz dwa zadania:
   - TaskA (priorytet 1) — co 200 ms zwiększa licznik i wyświetla go,
   - TaskB (priorytet 3) — co 1000 ms wyświetla komunikat „1234” i trzyma go przez 500 ms.
3. Każda operacja na wyświetlaczu odbywa się pod muteksem. TaskB zwalnia mutex dopiero po 500 ms wyświetlania komunikatu.
4. Zaobserwuj poprawne działanie: komunikat „1234” jest widoczny w całości przez 500 ms.

```
xSemaphoreTake(dispMutex, portMAX_DELAY);
wyswietl(1234);
vTaskDelay(pdMS_TO_TICKS(500));                // wyświetlacz zajęty
xSemaphoreGive(dispMutex);
```

Etap B — bez muteksu

5. Usuń wywołania Take/Give i zaobserwuj, co się dzieje z komunikatem TaskB.

Etap C — inwersja priorytetów

6. Przywróć mutex. W TaskA wydłuż sekcję pod muteksem: po wpisaniu licznika wykonaj 100 ms aktywnego oczekiwania (pętla na micros()).
7. Dodaj zadanie TaskM (priorytet 2), które co 2 s wykonuje 1 s aktywnych obliczeń i nie korzysta z wyświetlacza.
8. W TaskB zmierz millis() przed Take i po Take; wypisuj czas oczekiwania na wyświetlacz przez Serial.
9. Zamień mutex na semafor binarny (xSemaphoreCreateBinary() + początkowe xSemaphoreGive()) i powtórz pomiar. Porównaj maksymalne czasy oczekiwania TaskB.

Do analizy:

- Dlaczego przy semaforze binarnym TaskB czasem czeka ponad 1 s? Który mechanizm to naprawia i jak?
- Co się stanie, jeśli TaskA zajmie wyświetlacz na bardzo długo?

Zadanie 4: Kolejka komunikatów — producent → konsument

1. Utwórz kolejkę typu uint8_t: queue = xQueueCreate(10, sizeof(uint8_t));
2. Utwórz zadanie ButtonProducer, które sprawdza przyciski S1–S3 i po wciśnięciu wysyła do kolejki numer przycisku.
3. Utwórz zadanie LedConsumer, które odbiera dane z kolejki (xQueueReceive) i zapala odpowiednią diodę na 300 ms; numer ostatniego przycisku pokazuje na wyświetlaczu.
4. Sprawdź w praktyce, że szybkie wciskanie przycisków tworzy „kolejkę zdarzeń”.

Rozszerzenie

5. Wydłuż obsługę w konsumencie do 1 s, a producent niech wysyła numer co 100 ms (automatycznie).
6. Producent wysyła z czasem oczekiwania 0: xQueueSend(queue, &n, 0). Zliczaj odrzucone wiadomości i wyświetlaj ich liczbę.
7. Wyznacz, po ilu wysłanych wiadomościach kolejka się zapełnia, i porównaj z obliczeniami.

Sprawozdanie

- Kody i opis działania zadań 1–4.
- Odpowiedzi na pytania „Do analizy”.
- Wyniki pomiaru czasów oczekiwania TaskB z zadania 3 (mutex a semafor binarny) i wniosek.
- Zapas stosu wszystkich zadań (uxTaskGetStackHighWaterMark) w końcowej wersji programu.